Sri Lanka Institute of Information Technology

SE1020 – Object Oriented Programming

Year 1 Semester 2 – 2026

Assignment: Project

Workload Distribution

Y1S2.IT.WD.05.02

Group 95

Name	Student ID
Janandith B.K.H.	IT25102300
Alwis L.A.H.	IT25103008
Harshani W N	IT25101154
Nimjaya G.S.	IT25102352
Ifran F. M.	IT25101991
Subasingha S.U.D	IT25100361

Project Title: Inventory and Stock Management System
Technology: Spring Boot, Spring Data JPA, MySQL, HTML/CSS/JS
Architecture: Four-Layer (Model, Repository, Service, Controller)

Component 01: Product Management

Assigned Member: IT25100361 — Subasingha S.U.D

Description:
Manages the product catalog of the inventory system, allowing the addition, modification, and removal of products. Each product is linked to a category and a supplier through foreign key references.

CRUD Operations:

- **Create:** Add a new product with details like name, category, supplier, quantity, and price, stored in the products table via REST API.
- **Read:** Retrieve all products or search for a specific product by its product ID.
- **Update:** Modify product details such as name, quantity, price, or reassign the category and supplier.
- **Delete:** Remove a product record from the system.

UI Components:

- Product Listing Page (products.html)
- Add New Product Form
- Edit Product Form
- Product Details View

OOP Concepts Applied:

- **Encapsulation:** The Product class stores all six fields as private with public getters and setters to control access.
- **Abstraction:** ProductRepository extends JpaRepository, hiding all SQL operations. ProductService provides business logic methods that the controller calls without knowing database details.

Component 02: Stock Transaction Management

Assigned Member: IT25102352 — Nimjaya G.S.

Description:
Manages the recording of all stock movements — items coming IN (received) or going OUT (dispatched). When a transaction is created, the system automatically adjusts the related product's quantity in the products table, demonstrating cross-module service coordination.

CRUD Operations:

- **Create:** Record a new stock transaction with product ID, type (IN/OUT), quantity, date, and notes, stored in the stock_transactions table.
- **Read:** View all stock transactions or search by transaction ID to review movement history.
- **Update:** Modify transaction details such as quantity, type, date, or notes.
- **Delete:** Remove a stock transaction record from the system.

UI Components:

- Stock Transaction Listing Page (stock.html)
- Record New Transaction Form
- Edit Transaction Form
- Transaction History View

OOP Concepts Applied:

- **Encapsulation:** The StockTransaction class keeps all six fields private, including transactionId, productId, type, quantity, date, and notes.
- **Abstraction:** StockService hides the business logic of adjusting product quantities from the controller. It uses @Transactional to ensure data consistency across both StockTransactionRepository and ProductRepository.

Component 03: Supplier Management

Assigned Member: IT25101154 — Harshani W.N

Description:

Manages supplier information for vendors that provide products to the inventory. Supplier records include contact details such as phone, email, and address. Products in the system reference suppliers through the supplier_id foreign key.

CRUD Operations:

- **Create:** Register a new supplier with details like name, contact number, email, and address, stored in the suppliers table.
- **Read:** View all suppliers or search for a specific supplier by supplier ID.
- **Update:** Modify supplier details such as contact information, email, or address.
- **Delete:** Remove a supplier record from the system.

UI Components:

- Supplier Listing Page (suppliers.html)
- Add New Supplier Form
- Edit Supplier Form
- Supplier Details View

OOP Concepts Applied:

- **Encapsulation:** The Supplier class stores five private fields (supplierId, name, contact, email, address) with controlled access through getters and setters.

- **Abstraction:** SupplierRepository extends JpaRepository, providing automatic CRUD without any SQL. SupplierService exposes simple methods to the controller while hiding database interaction details.

Component 04: Category Management

Assigned Member: IT25103008 — Alwis L.A.H.

Description:

Manages product categories that group related items together (e.g., Electronics, Office Supplies, Furniture). Products reference categories through the category_id foreign key, allowing organised classification of inventory items.

CRUD Operations:

- **Create:** Add a new category with a name and optional description, stored in the categories table.
- **Read:** View all categories or search for a specific category by category ID.
- **Update:** Modify category details such as name or description.
- **Delete:** Remove a category record from the system.

UI Components:

- Category Listing Page (categories.html)
- Add New Category Form
- Edit Category Form
- Category Details View

OOP Concepts Applied:

- **Encapsulation:** The Category class stores three private fields (categoryId, name, description) with getters and setters enforcing controlled access.
- **Abstraction:** CategoryRepository extends JpaRepository with no custom SQL queries needed. CategoryService provides clean business methods that abstract away all data access logic from the controller.

Component 05: User and Admin Management

Assigned Member: IT25102300 — Janandith B.K.H

Description:

Manages user accounts and administrative roles in the system. This module implements the full OOP inheritance hierarchy using an abstract Employee base class with Admin and StaffMember subclasses. It uses JPA's SINGLE_TABLE inheritance strategy, storing all user types in a single users table differentiated by a discriminator column.

CRUD Operations:

- **Create:** Register a new user (Admin or Staff) with name, email, password, role, and type-specific fields (department for Admin, section for Staff), stored in the users table.
- **Read:** View all users or search for a specific user by user ID. The system returns the correct subclass type based on the discriminator value.
- **Update:** Modify user details such as name, email, password, role, department, or section.
- **Delete:** Remove a user account from the system.

UI Components:

- User Listing Page (users.html)
- User Login Page (login.html)
- Admin Dashboard (dashboard.html)
- User Registration Form
- User Profile Edit Form

OOP Concepts Applied:

- **Encapsulation:** All fields in Employee, Admin, and StaffMember classes are private with public getters and setters.
- **Inheritance:** Admin and StaffMember extend the abstract Employee class, reusing shared fields (userId, name, email, password, role) while adding their own specific fields.
- **Polymorphism:** UserRepository uses the parent type Employee, allowing it to store and retrieve both Admin and StaffMember objects through a single repository interface.
- **Abstraction:** UserService and UserRepository hide the complexity of handling multiple user types behind simple CRUD method calls.

Component 06: Reports and Low Stock Alerts

Assigned Member: IT25101991 — Ifran F.M.

Description:

Manages the generation, storage, and export of inventory reports. This module performs live analytics by aggregating data from products, stock_transactions, and users tables. It calculates total inventory value, identifies low-stock products, and provides stock movement summaries. Reports can also be exported as TXT files.

CRUD Operations:

- **Create:** Generate a new report with a title, content, and the ID of the user who generated it, stored in the reports table.
- **Read:** View all reports or retrieve a specific report by report ID. Additional analytics endpoints provide live inventory statistics.
- **Update:** Modify report details such as title or content.
- **Delete:** Remove a report record from the system.

UI Components:

- Report Listing Page (reports.html)
- Generate New Report Form

- Report Details View
- Low Stock Alerts Panel
- Export Report to TXT

OOP Concepts Applied:

- **Encapsulation:** The Report class stores private fields (reportId, title, content, generatedBy, generatedDate) with getters and setters.
- **Abstraction:** ReportService hides complex cross-module analytics behind simple method calls. It injects ProductRepository and StockTransactionRepository from other modules to aggregate data, while the controller only calls high-level service methods.
